

① Median finding algorithm

Lecture 4

①
Date - 29/08/2022

• looked at algorithm for setting mainly quick sort.

• The way quick sort works

- pick a pivot

- Compare each element in the array with the pivot

- Divide the array into two parts

 - elements less than the pivot

 - elements greater than the pivot.

- Recursed on both the parts to sort them

- Finally put them together to get the larger sorted array

• The time taken by the quicksort crucially depends on the position of the pivot element in

the array

which we call median

- If the middle element chosen as pivot the quicksort gives best performance

- If the median is chosen ~~fast~~ always chosen first then perhaps the quicksort algorithm works in $O(n \log n)$ time.

So our objective is to find

- given an array of size n
- find the median in time linear i.e. $O(n)$

Few definitions

rank of an element: The rank of an element is its position in the ~~sorted~~^{sorted} array.

- So given an array and an element
 - sort the array and on the position of that element in the sorted array is its rank.
- we assume that all the elements in the array are distinct.
 - all above will work even if all elements are not distinct.
 - This is just for simplicity

The way rank is defined, the median of an array is an element of rank $\lfloor n/2 \rfloor$

Our objective is to find 'median' of an array and it should be fast.

(3)

input: Array A
output: Median

clearly it can be done
by $O(n \log n)$ time -
- sorting the array
- picking the middle
element

but we need faster algorithm, but how?

Can we apply divide and conquer?

but how to divide the array

Suppose you have somehow find the approximate median:

approximate median: an element of rank greater than $n/4$ and less than $3n/4$ i.e. if x is the element,

$$\frac{n}{4} \leq \text{rank}(x) \leq \frac{3n}{4} \quad n \text{ is the size of the array}$$

↓

approximate median $\Rightarrow$ They are not quite the middle element but they roughly sit around the middle element

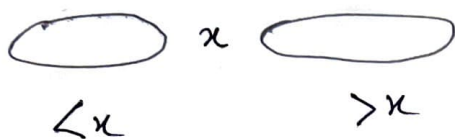
now question is if someone gives the approximate median, does this help to find the exact median?

The answer is yes it may help, in the following sense -

Suppose x is the approximate median



x : approximate median.



This is my array A

Let's not focus ~~on~~ goal. i.e. finding the exact median

- x is approximate median
sometimes we can know ~~about~~ it
- what we could do, split the array into less than x and greater than x
- once we do this we know where the median falls

- if the rank of $x < n/2$ then median falls on the right

- on the other hand if $\text{rank}(x) > n/2$, it falls on the left

- good thing about the partition is that both the partition has size at least $n/4$

as ~~no~~ $\text{elems} > n/4$ x is $n/4$ at least

" " $< x$ is $n/4$

so if we can get rid of one part, say the smaller part then the bigger part is of size at least $3n/4$.

size at least $3n/4$.

we have shown the size of the array by a constant factor.
does this help?

Actually it does

Suppose we can find the Approximate median somehow in linear time, so let's see how much time does it take to find the exact median.

What we would like to do is this, suppose we have an array of size n . So

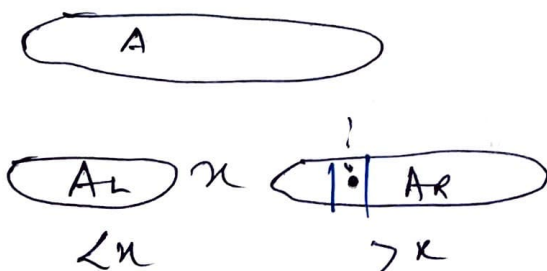
we have

$$T(n) = n + T\left(\frac{3n}{4}\right) \\ \approx O(n)$$

approximate median tree
n time

by suppose we can do this, the recurrence gives us $O(n)$
we can open out few terms and check.

There is a problem. we want to find the median of the big array A . Somehow get the approximate median in $O(n)$ time. (still don't know how this is done)



$$T(n) = n + T\left(\frac{3n}{4}\right)$$

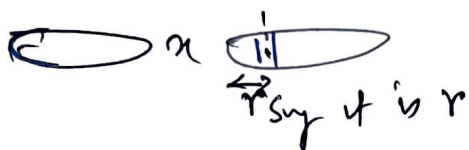
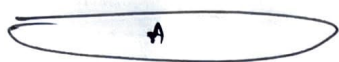
The element could sit not exactly in the median
The element that I want to find depends on the rank of the element
The left hand side of the array.

Let's make the A_L & A_R

lets say

$|A_L| < \frac{n}{2}$ w.l.g. what we want is median of
The entire array

Now what the rank of ~~arr~~ of the element in AR

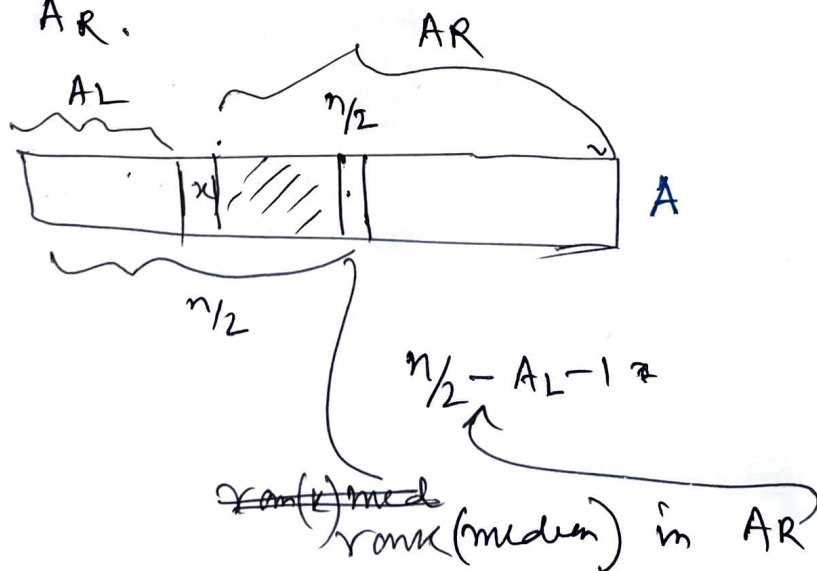


we have: $r + 1 + A_L = \boxed{\boxed{\frac{n}{2}}}$

$$r = \left\lfloor \frac{n}{2} \right\rfloor - A_L - 1$$

So here r is rank of the median

of ~~arr~~ A in AR.



So when we recurse we don't want The median
in A, we are actually losing an element of
rank $\frac{n}{2} - A_L - 1$ in AR.

which need not be the
middle element in AR

So when we recurse we are not trying to find the middle element rather we are trying to find an element of rank $\boxed{A_L - 1 - r}$

we want to find an element of particular rank.

So, why not find a procedure to find an element of any rank.

pay careful attention here.

- we started off to find the median.
- we tried to find a recursive procedure
- along the way we encountered problem where we actually want to find the element of some other rank. not just rank $n/2$ but some other rank.
- So this seems to be a more general problem finding the median is a special case of the problem
- sometime it is easier to solve a general problem than a specific problem

The problem we would like to solve is ~~not~~ just finding the median but an element of any rank.

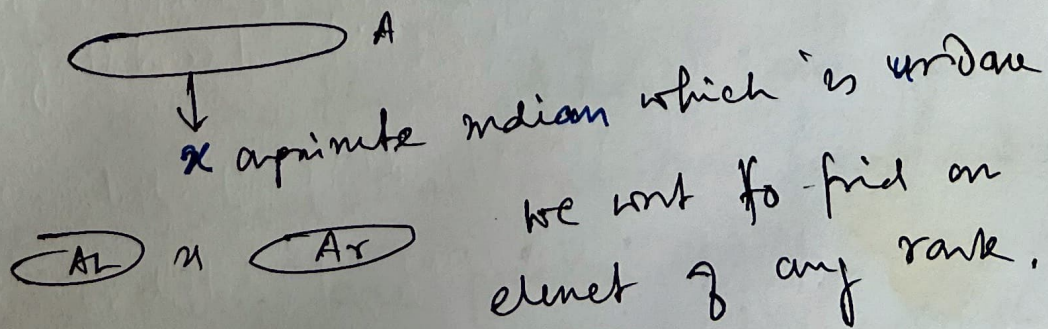
So the input is: Array A, r

output: an element of any rank r in A

So if $r = n/2$ ~~then~~ we get the median

So this is the more general problem than the one we started with. and this is what we would like to solve. The ~~problem~~ reason this problem ~~was~~ came up is that during recursion we tried ~~with~~ with the median and ~~but~~ eventually we end up with this problem: find element of any rank.

So now there is no problem with the recursion now.



if r is less than size of A_L then we

⑨ recursive of AL 'if $|A_L| > |A_R|$ recursive on AR

So as long as we can find the approximate median we can find the element of any rank.

Now you can write down the recurrence relation and see that ~~Now the recursion looks like 'if we can find the approximate median in linear time then you can find an element of any rank in linear time.'~~

So how do we find the approximate median? This seems to be as difficult as finding the exact median of any rank.

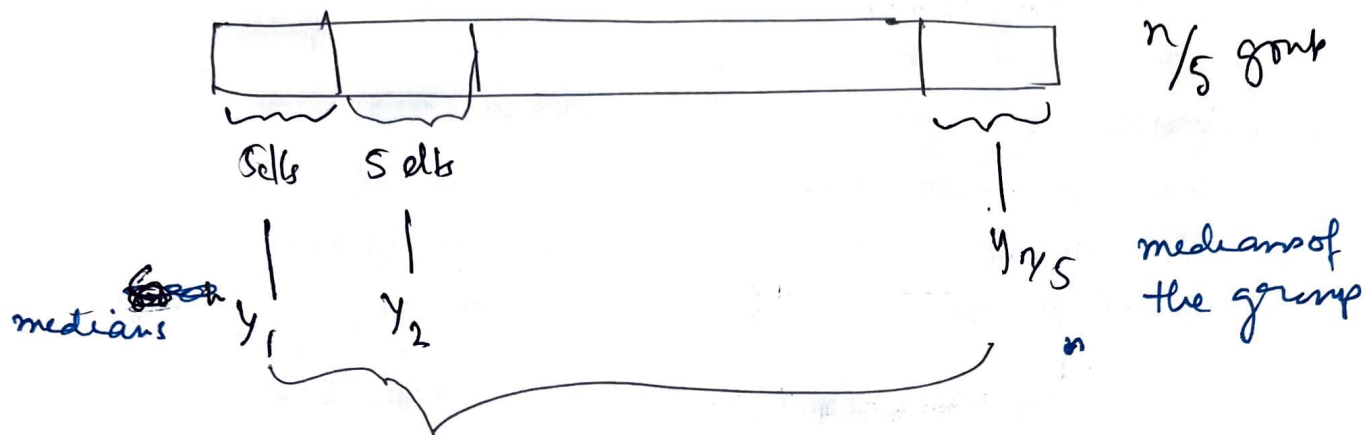
So the solution that I am going to present is really smart. So the way we

I can not tell how exactly people came up with this, sometime we just have to be really clever and come up with the solution.

There is no real recipe for finding the algorithm all the time. Although there are few general recipe that one can follow but often this thing are problem specific dependent. That you think about it and think about it and somehow something clicks in your mind and come up with the algorithm.

The way we find approximate median is
So here is the algorithm that works.

Take the array A , split the element in groups
of 5, so we have $n/5$ groups in A and each
of them has 5 elements.



- Find the median in each group.
- Now ~~approximate~~ ^{the exact} median ~~that is pulled out~~ of these $n/5$ elements would give us the approximate median

There are no qubits now.

- Why should this be an ~~approximate~~ ^{approximate} median
- We started with finding the exact median
- we ended up finding the approximate median
- and again we are finding the exact median

To find this approximate median we find these

$y_1, y_2, \dots, y_{n/5}$ and again we are finding the exact median. What is going on? Well, the second thing not so serious.

→ initially we ~~are~~ ^{strike out} finding the median of n elements

— we then find the median of $n/5$ elements so the this size has reduced and we can recurse on them.

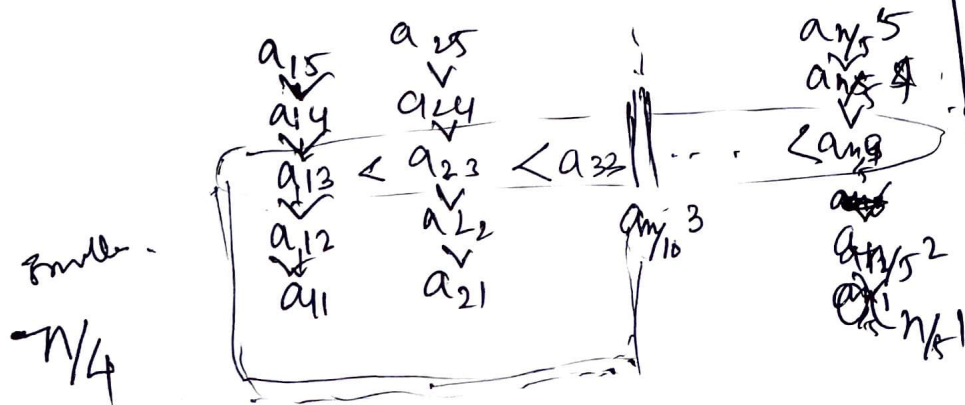
exact median of this $n/5$ element ~~on that~~ we want to find out, we just do it by recursion

I still have to convince you that the exact median of this $n/5$ element would be the approximate median. To see this let me arrange these 5 elements in each group in decreasing order.

upper elements

next thing ~~we~~ I am going to do is to look at these middle elements, i.e. median of these groups, and merge this group in sorted order to work the right

one of these may not have 5 elements



find element (r, A) // finds an element of rank r in A

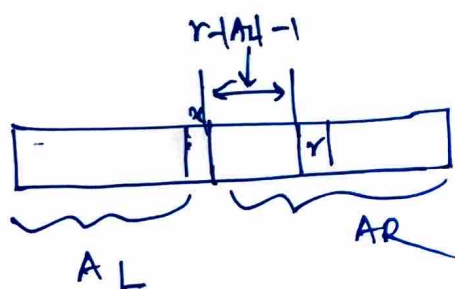
- partition A into $|A|/5$ groups of size 5
 - find the median of each array group and put them in an array B . size of B is $|A|/5$.
 - find element ($|B|/2, B$)
- Finds approximate median
This should return an element call it as x .

• Partition A into A_L & A_R
(elts $< x$ are in A_L &
elts $> x$ are in A_R).

Now there are three cases

• $|A_L| = r-1$ then return x .

• $|A_L| > r-1$
find element (r, A_L) { elements of rank r into in A_L



• $|A_L| > r-1$
find element ($r - |A_L| - 1, A_R$)

* The element of rank r in A is also an element of rank r in A .

Now how do we analyse the algorithm. Our expectation is that this procedure runs in linear time. Let us analyse and see.

The easiest way to analyse an algorithm is to write down the algorithm, perhaps the code for the algorithm. Then look at each step of the algorithm and figure out how many times each step is executed. ~~Then~~ One of the steps in the algorithm could be recursive steps. ~~and~~ It may also have loops. Those outside loops executed once. For the recursive call you need to estimate the size of the input. For instance recursive steps may be on size ~~size~~ size n initially then in the next step size could be $n/5$ and so on. For loops you need to estimate how many times the loop is executed.

So for our algorithm. ~~Let us~~ suppose $T(n)$ is the time taken on an array of size n .

$$T(n) = C_1 n + T\left(\frac{n}{5}\right) + n + T(\quad)$$

median of each group. There are only 5 elements, we can even do a bubble sort which takes 25 comparisons. So it is some constant time n .

The array is of size $n/5$ for partitioning each element is compared

Recursion on any after finding the approximate median.

either we recurse on A_L or A_R depending on where the approximate median x lies.

Notice that size of $|A_L| \geq \frac{n}{4}$ &

$$|A_R| \geq \frac{n}{4}$$

$$\downarrow$$

$$|A_L| \leq \frac{3n}{4}$$

So size of $|A_L|$ is most $\frac{3n}{4}$

because size of $|A_R|$ is at least $\frac{n}{4}$.

Similarly we can conclude that $|A_R| \leq \frac{3n}{4}$

So whichever side we recurse the maximum size of the array could be $\frac{3n}{4}$ so we can

rewrite the recurrence as

$$T(n) \leq C_1(n) + T\left(\frac{n}{5}\right) + \underline{n} + T\left(\frac{3n}{4}\right)$$

median

recurse to
find in
exact median

partitions

last recursion

approximate median

elements of
any rank
 σ_i

$$\leq (C+1)n + T\left(\frac{n}{5}\right) + T\left(\frac{3n}{4}\right)$$

